TELECOMM USECASE-----SPRINT1

DIM_TOWER IMPLEMENTATION:

It is created for the purpose of implementing SCD1 and to get distinct TowerID's available.

SCD1 is used here to handle the overwriting of data whenever the src_input updates.

Why Not SCD2?

Because SCD2 handles history of data.Here we do not require any history of data.So we are going with SCD1 which will overwrites the existing records in existing row with updated records.

Source Data:

Clipboard Font

POSSIBLE DATA LOSS Some features might be lost if you save th

C12

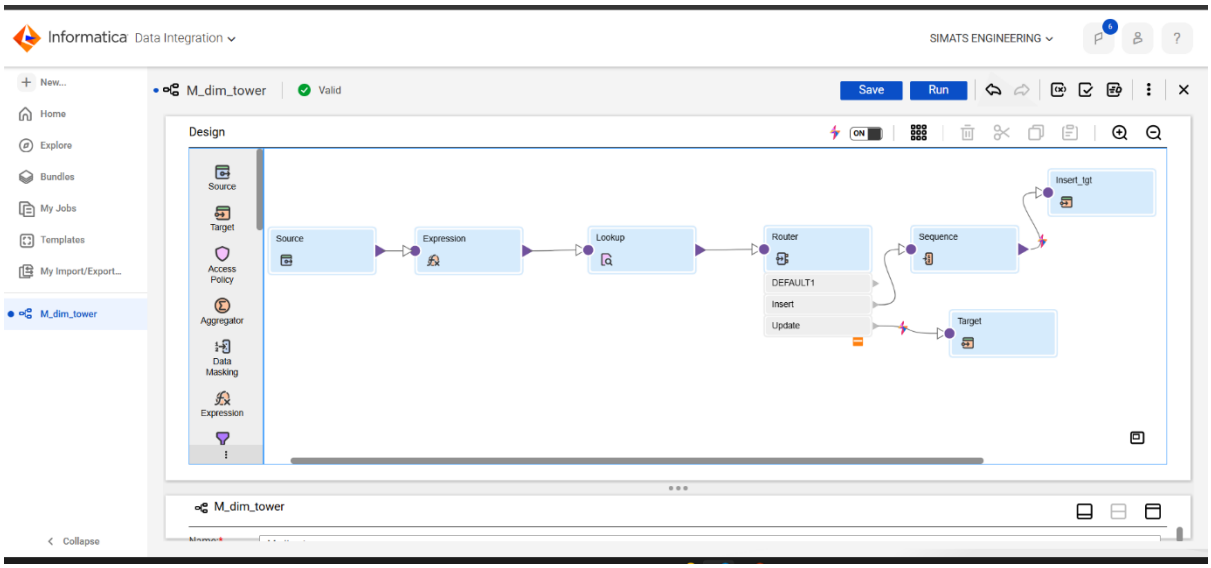
	A	B	C	D	E	F
1	TowerID	Region	City			
2	T001	East	Mumbai			
3	T002	South	Chennai			
4	T003	West	Chennai			
5	T004	West	Hyderabad			
6	T005	East	Mumbai			
7	T006	North	Hyderabad			
8	T007	West	Chennai			
9	T008	South	Mumbai			
10	T009	North	Bangalore			
11	T010	South	Chennai			
12						
13						
14						
15						
16						

Implementation Steps:

/----→Sequence-→Insert_tgt

SRC-→Expression→LookUp→Router-→

\----→Update_tgt



Expression Usage:

Expression is used for data cleaning by trimming the extra spaces and handling null values.

Expression Used:

1.Clean_TowerID = LTRIM(RTRIM(TowerID))

2.Clean_Region = IIF(ISNULL(Region), 'UNKNOWN', UPPER(LTRIM(RTRIM(Region))))

3.Clean_City = IIF(ISNULL(City), 'UNKNOWN', INITCAP(LTRIM(RTRIM(City))))

Lookup Transformation:

It is Used for looking and fetching required rows from tgt table

Lookup Condition Used:

Src.TowerID=lkp.Tower.ID

Router Transformation:

It is used for splitting the data as per required conditions. Here I had splitted data into two paths

1.Insert-→ ISNULL(TOWER_KEY)

2.Update-→NOT ISNULL(TOWER_KEY)

AND (

Src_clean_Region != REGION OR

Src_Clean_City != CITY

)

Sequence Generator:

It is used for giving key numbering for newly inserted records with increment as 1 and intial as 1 with mapping field NEXTVAL to map in insert_tgt table to generate the key.

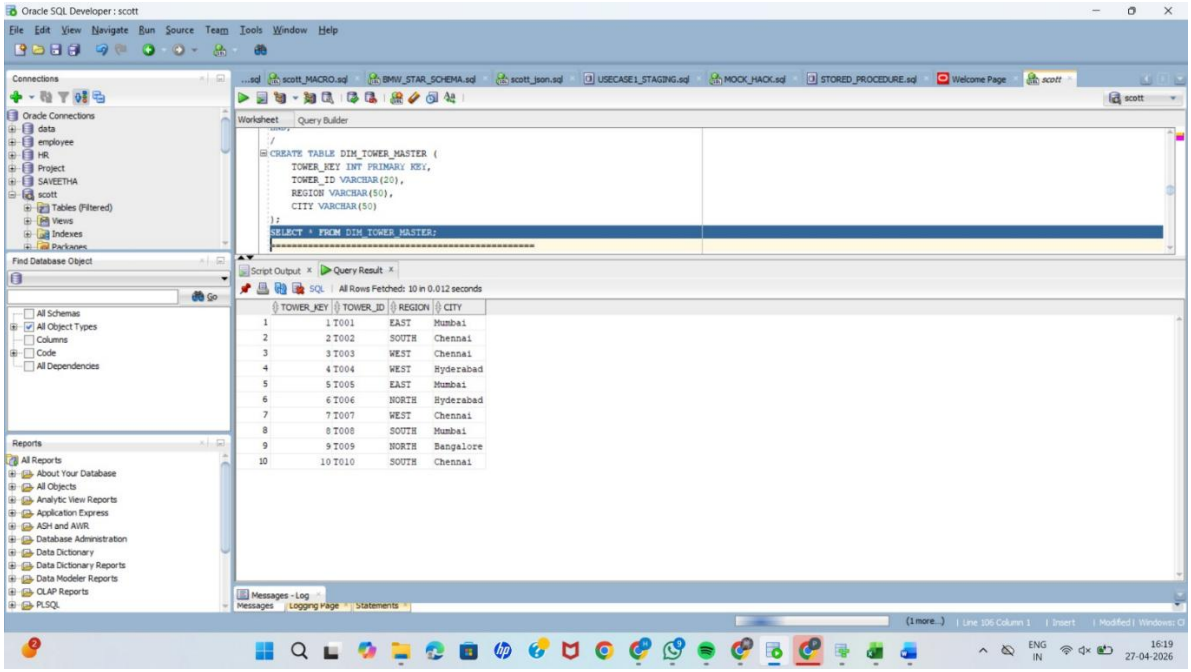
Insert_tgt:

This table contains inserted records and manages new insertions

Update_tgt:

This table manages the updations of records by overwriting them as per input.

Result Table:



USECASE IMPLEMENTATION

Daily Call Volume Summary

KPIS:

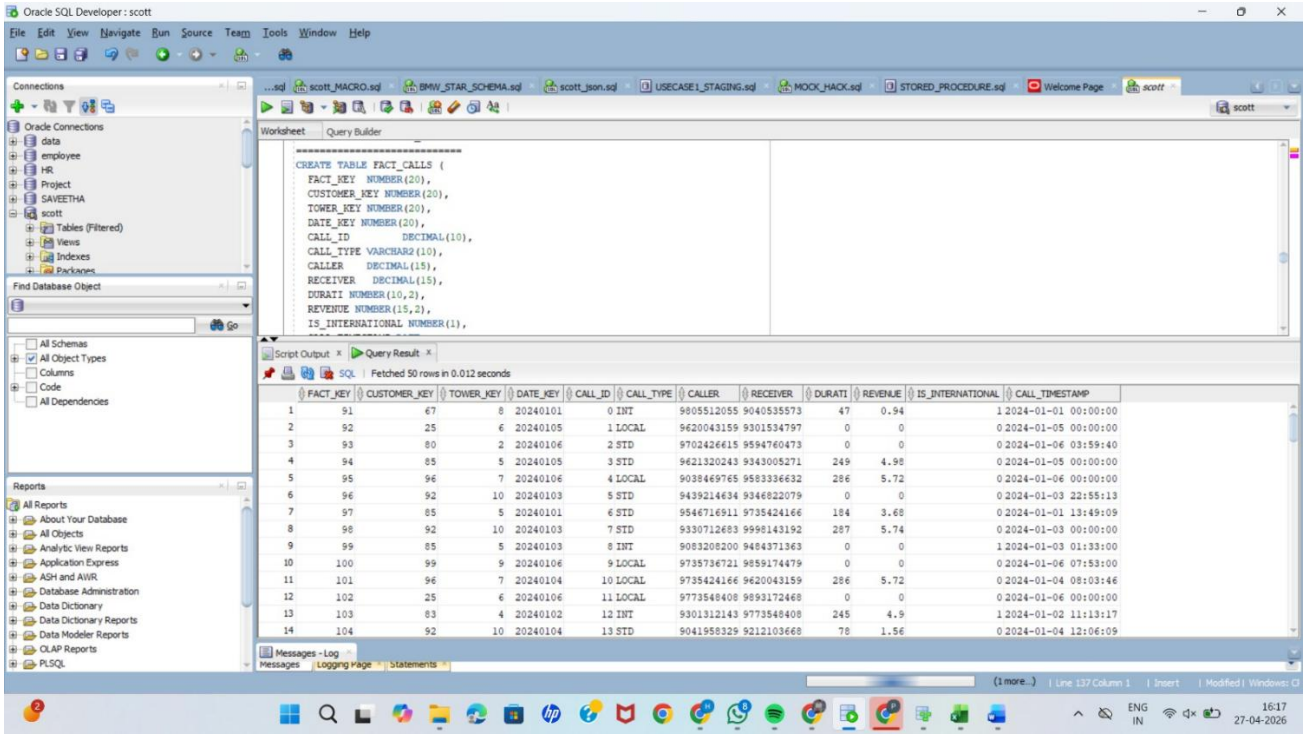
1.Unique Customers Called

2.Avg Calls per active customer

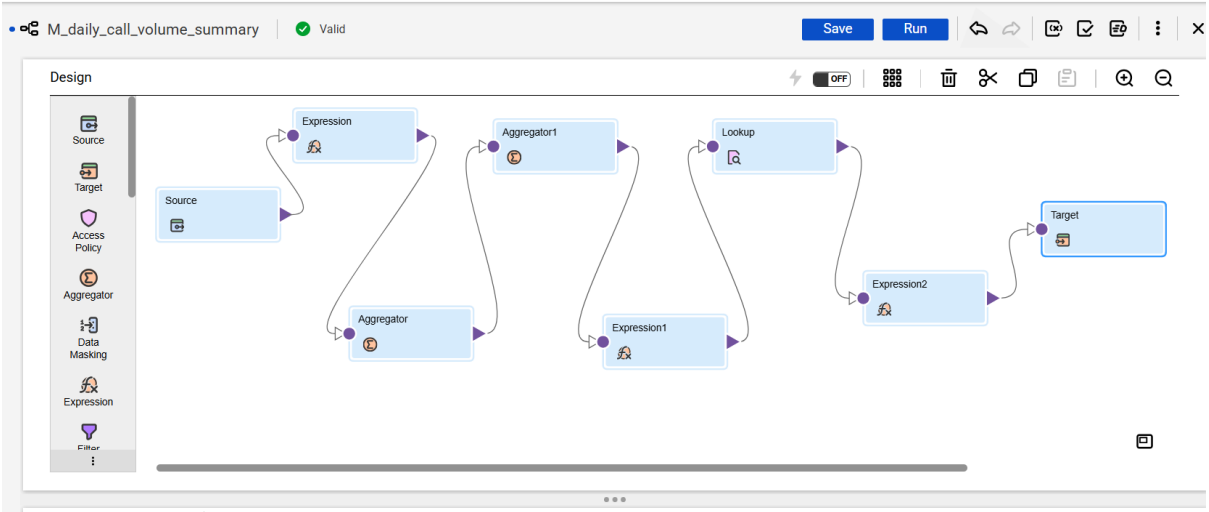
3.Peak call duration

4.Daily Call Volume Trend

Source Table:



Implementation Plan:



Expression:

We are using this to convert the date which is in timestamp to date format like ‘DD-MM-YYYY’.

Expression Used:

Expr_call_date= TO_DATE(TO_CHAR(CALL_TIMESTAMP, 'DD-MM-YYYY'), 'DD-MM-YYYY')

In this TO_CHAR(CALL_TIMESTAMP, 'DD-MM-YYYY'-→converts timestamp to string so it removes the time part. Then To_Date makes the string to date conversion.

Aggregator:

Groups expr_call_date and caller

Aggregator condition:

Totalcall=count(call ID)

Max_duration=max(duration)

Aggregator 2:

Groups expr_call_date

Aggregator condition:

New_uniq_cust=count(caller)

Expression 1:

Undergoes kpi caluclations

Calculates Avg_calls_per_cust=
IIF(new_unique_cust = 0,

```
   0,
   TOTALCALLS / new_unique_cust
)
```

Prev_date= ADD_TO_DATE(expr_call_date, 'DD', -1)

This is used for get previous date by applying 'DD' with -1.

Lookup Transformation:

Looks through the tgt table and fetches

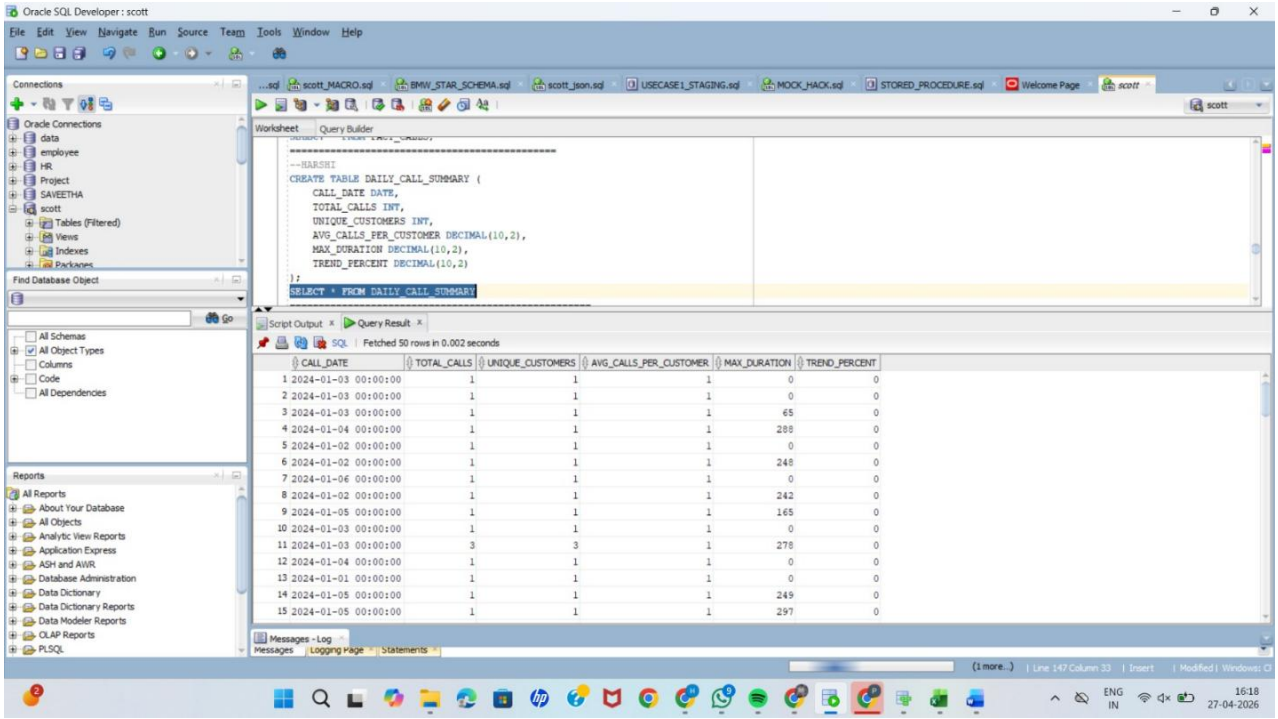
Call_date =prev_date

Looks through prev date and compares

Expression2:

Calculates Trend_percent:

```
IIF(
  ISNULL(TOTAL_CALLS) OR TOTAL_CALLS = 0,
  0,
  ((expr1_TotalCalls - TOTAL_CALLS) / TOTAL_CALLS) * 100
)
```

Connect the transformations with target and go through the validation and run the mapping.

Result Table: